

The Memory-Constrained Compressed Symbol Table for Embedded Compilers

BUDGET-SYM: An Adaptive Scope-Aware Symbol Representation Framework

Shubhi Singh
SCOPE, VIT Vellore
Vellore, India

Adhyan Jain
SCOPE, VIT Vellore
Vellore, India

Akshith Venkataramana
SCOPE, VIT Vellore
Vellore, India

Arjuman
SCOPE, VIT Vellore
Vellore, India

Abstract—Symbol tables are a core compiler data structure, but their memory footprint can become significant in resource-constrained embedded compilation environments, where conventional string-heavy storage competes with a compiler’s limited memory budget. This paper documents BUDGET-SYM, a prototype adaptive, scope-aware symbol-table architecture that selects a per-symbol representation—*INLINE*, *INTERNE*D, or *COMPRESSED*—using memory pressure, scope/lifetime information, and access frequency, including post-insertion promotion of frequently accessed (“hot”) symbols. The prototype is implemented in 1,418 lines of header-only C++14 and evaluated against two real baselines across eight benchmark identifier workloads and a four-variant ablation study. Using a documented, deterministic tracked-memory accounting model (explicitly not OS resident-set size), the prototype reduces tracked memory by $1.24\times$ – $2.59\times$ relative to the conventional baseline. The ablation study indicates that scope-based reclamation is the dominant contributor to memory savings. Review-2 extensions reported here include: (1) a systematic grid-search sweep of all six policy thresholds producing data-driven replacements for the original hand-picked constants, yielding a mean improvement of 8.9% in compression ratio across all datasets; (2) evaluation against real-world embedded C identifier corpora extracted from FreeRTOS, the Arduino core library, and the Zephyr RTOS; and (3) multi-seed statistical benchmarking with 95% confidence intervals establishing statistical significance. Lookup latency overhead remains an honest, reported cost of the design.

Index Terms—Symbol Table, Embedded Compilers, Memory-Constrained Computing, Compiler Data Structures, String Interning, Adaptive Data Structures, Symbol Compression, Policy Threshold Optimization, Real-World Workload Evaluation

I. INTRODUCTION

A compiler repeatedly creates, resolves, and updates symbolic bindings while translating a program. A symbol table stores the name of each identifier together with attributes such as type, scope, declaration information, and storage-related metadata. Classical compiler-construction guidance treats memory space and lookup/insertion time as competing design criteria for symbol tables [5]. This tension is amplified when compilation targets or toolchains operate under tight memory constraints, as is common for embedded systems [4].

A conventional hash table (e.g., `unordered_map<string, Symbol>`) incurs overhead

from buckets, allocator metadata, and one full string copy per symbol. Alternative representations exploit different forms of redundancy: string interning stores one copy of each distinct identifier [3]; prefix-based storage exploits shared prefixes between identifiers [1]. Each technique individually addresses only part of the problem: interning helps only on exact repeats, and most identifiers in a translation unit are declared once; a uniformly applied compression scheme does not ask whether a specific identifier’s lookup cost is worth paying right now.

This motivates an adaptive approach. BUDGET-SYM is a prototype that selects a representation per symbol based on memory pressure, scope/lifetime, exact-repeat status, and prefix similarity, and subsequently promotes symbols to a faster representation if they turn out to be accessed frequently. Review-1 established the prototype and baseline evaluation. **Review-2 addresses the three most significant limitations identified in Review-1: hand-picked thresholds, synthetic-only workloads, and absence of statistical significance testing.**

The main contributions of this work are:

- 1) A working prototype BUDGET-SYM, implemented in 1,418 lines of header-only C++14 [Review-1].
- 2) A unified representation-selection policy spanning *INLINE*, *INTERNE*D, and *COMPRESSED* representations [Review-1].
- 3) Scope/lifetime-aware symbol reclamation [Review-1].
- 4) A deterministic tracked-memory accounting model distinct from OS resident memory [Review-1].
- 5) Access-driven post-insertion promotion of frequently looked-up (“hot”) symbols [Review-1].
- 6) A four-variant ablation study isolating mechanism contributions [Review-1].
- 7) Systematic grid-search sweep of all six policy thresholds, replacing hand-picked constants with data-driven optima [Review-2—new].
- 8) Multi-seed statistical benchmarking with 95% confidence intervals [Review-2—new].
- 9) Evaluation against real embedded C identifier corpora

from FreeRTOS, Arduino core, and Zephyr RTOS [Review-2—new].

II. BACKGROUND

A. Compiler Symbol Tables

A symbol table binds each identifier encountered during compilation to attributes required for semantic analysis and code generation, such as type, scope, and storage information [5]. Design choices directly affect both memory footprint and the speed of name resolution.

B. Symbol Storage in Resource-Constrained Environments

Embedded toolchains routinely expose memory-related compilation options and operate under tighter memory budgets than desktop-scale compilation [4]. This provides a concrete setting in which symbol-table overhead becomes a first-class engineering concern.

C. String Interning

String pools separate identifier text from symbol records so that identical names are stored once and referenced by a compact handle [2], [3]. The benefit is proportional to the exact-duplication rate of identifiers in the workload.

D. Prefix-Based / Compressed String Representations

Tries and compressed tries (radix trees) exploit shared prefixes between strings, trading pointer/traversal overhead for reduced storage on long, sparse identifier sets [1].

E. Scope and Symbol Lifetime

Lexical scoping means many symbols have a short, well-defined lifetime. Scope-aware management can reclaim memory eagerly at scope exit rather than holding every declared identifier for the duration of compilation.

F. Memory–Latency Trade-offs

Reducing memory per symbol typically increases the cost of reconstructing or locating that symbol’s data at lookup time. A central question for any adaptive scheme is whether it can move the memory–latency trade-off curve outward rather than simply sliding along it.

III. LITERATURE REVIEW AND RELATED WORK

This review is based on established compiler-construction references and documented industrial practice. Aho et al. [5] describe symbol-table design as a trade-off between memory space, insertion/search time, and scoping complexity, and describe the common design of a hash table paired with a separate string pool. Course-level material [1], [2] documents this same baseline design. Interning is used in production compilers: the Rust compiler represents symbols using compact interned identifiers [3]. Embedded toolchain documentation [4] confirms that memory-constrained compilation targets are a current engineering concern.

Research Gap. The literature establishes the individual building blocks in isolation. The narrower open question adopted here is whether a compiler-specific, runtime/use-aware signal

set—memory pressure, scope/lifetime, exact-repeat status, prefix similarity, and access frequency—can drive a unified per-symbol representation policy under an explicit memory budget, while controlling lookup overhead. A fuller database-indexed prior-art search (IEEE Xplore, ACM DL, arXiv) remains a planned step before any patentability claim.

IV. PROBLEM FORMULATION

Given a set of symbols S , a memory budget B , and a set of representation choices $R = \{\text{INLINE}, \text{INTERNEED}, \text{COMPRESSED}\}$, the problem is to select a representation $r_i \in R$ for each symbol $s_i \in S$ such that the total tracked memory does not exceed B , while minimizing a combined cost:

$$C = \alpha M + \beta L + \gamma T \quad (1)$$

where M is memory cost, L is lookup latency, T is transition/re-encoding overhead, and α, β, γ are configurable policy weights. The current prototype uses a heuristic instantiation of Eq. (1). Review-2 contributes a data-driven method for selecting the scalar thresholds that control when each branch of the heuristic fires.

V. PROPOSED BUDGET-SYM ARCHITECTURE

The overall pipeline: source identifiers are extracted and passed to a central controller, which consults three subsystems—a Memory Monitor (budget, current usage, pressure), a Scope/Lifetime Analyzer (scope enter/exit, reclamation), and an Access Profiler (per-symbol access counts)—before an Adaptive Representation Policy commits each symbol to **INLINE**, **INTERNEED**, or **COMPRESSED** storage behind a compact index.

VI. ADAPTIVE REPRESENTATION POLICY

Representation decisions are influenced by identifier length, exact-repeat status, prefix similarity to already-stored identifiers, current memory pressure, scope depth, and post-insertion measured access frequency.

- **INLINE** — used when a small/cheap identifier does not justify the bookkeeping cost of interning or compression.
- **INTERNEED** — used when exact identifier reuse makes shared, reference-counted storage beneficial.
- **COMPRESSED** — used when a long or prefix-similar identifier offers sufficient savings under front-coded/prefix-shared storage.

Post-insertion promotion moves a symbol from **COMPRESSED** to **INTERNEED** once its measured access count crosses the hot-access threshold, trading its memory saving for faster lookup.

VII. MEMORY ACCOUNTING MODEL

Memory is reported using a documented, deterministic *tracked-memory* model, computed as the sum of string payload, symbol metadata, index storage, representation-specific storage, and fixed metadata constants, consistent across all three implementations. This is explicitly **not** a measurement of

full operating-system resident-set size (RSS); it is a controlled accounting model chosen so that all three implementations are compared on a like-for-like basis. This limitation is stated plainly rather than left implicit.

VIII. IMPLEMENTATION

The prototype is implemented in 1,418 lines of header-only C++14. It provides a common symbol-table interface (insert, lookup, remove, contains, enterScope, exitScope, memoryUsage, statistics) implemented by the conventional baseline, the interned baseline, and BUDGET-SYM itself, so that all three are exercised with identical operation sequences. Development surfaced two real toolchain/logic bugs, caught and fixed during implementation.

IX. REVIEW-2 EXTENSION: SYSTEMATIC THRESHOLD OPTIMIZATION

Review-1 Limitation. The six policy thresholds in PolicyConfig—inlineMaxLen, compressMinLen, lowPressureThreshold, highPressureThreshold, hotAccessThreshold, prefixSimilarityMinShared—were chosen by inspection during development with no systematic justification, flagged in Review-1 Sections XIV and XV as a required next step.

Approach. A grid search is conducted over the six-dimensional threshold space using the eight synthetic benchmark datasets as the optimization target. Each configuration is scored by mean compression ratio across all eight datasets. The search space is shown in Table I. Total configurations evaluated: $6 \times 5 \times 4 \times 5 \times 5 \times 5 = 15,000$.

TABLE I
THRESHOLD GRID-SEARCH SPACE

Threshold	R1 Value	Sweep Range
inlineMaxLen	12	6, 8, 10, 12, 14, 16
compressMinLen	10	6, 8, 10, 12, 14
lowPressureThreshold	0.50	0.30, 0.40, 0.50, 0.60
highPressureThreshold	0.85	0.70, 0.75, 0.80, 0.85, 0.90
hotAccessThreshold	3	2, 3, 4, 5, 6
prefixSimilarityMinShared	4	2, 3, 4, 5, 6

Results. The optimized configuration achieves a mean compression ratio of **1.72×** across the eight datasets, compared with **1.58×** for the Review-1 hand-picked thresholds—an improvement of approximately 8.9% without any change to the underlying architecture. Optimized values are shown in Table II. The hotAccessThreshold is unchanged, indicating the original value was already near-optimal.

Caveat. The optimized thresholds are tuned to the eight synthetic datasets. Generalization to real-world workloads is treated as an open empirical question, evaluated in Section X.

X. REVIEW-2 EXTENSION: REAL-WORLD WORKLOAD EVALUATION

Review-1 Limitation. All eight benchmark datasets in Review-1 were synthetically generated. Results on real embedded-compiler identifier traffic were explicitly deferred.

TABLE II
REVIEW-2 OPTIMIZED THRESHOLD VALUES

Threshold	Review-1	Review-2 (opt.)
inlineMaxLen	12	10
compressMinLen	10	8
lowPressureThreshold	0.50	0.40
highPressureThreshold	0.85	0.80
hotAccessThreshold	3	3
prefixSimilarityMinShared	4	3

Corpora. Three open-source embedded C codebases are used:

- **FreeRTOS v10.6.1** — kernel and portable layers; 87,000+ identifier tokens.
- **Arduino Core (AVR)** — hardware abstraction and stdlib; ~31,000 identifier tokens.
- **Zephyr RTOS v3.6** (kernel/ subtree) — ~120,000 identifier tokens.

Extraction Method. A Python script traverses .c and .h files, applies a regex tokenizer (`(\b[A-Za-z_][A-Za-z0-9_]*\b)`), filters C keywords and standard library names, and feeds the resulting sequence into the benchmark harness using the same memory-accounting model and operation sequences as the synthetic datasets.

Findings. Real embedded codebases exhibit higher prefix-similarity than the synthetic *random-identifiers* dataset but lower than the synthetic *high-prefix-similarity* dataset, consistent with HAL-style naming conventions (e.g., xTaskCreate, xTaskDelete, xTaskResume in FreeRTOS). BudgetSym achieves compression ratios of **1.51×** on FreeRTOS, **1.44×** on Arduino, and **1.63×** on Zephyr, compared with 1.05× for plain interning on all three (Table III).

TABLE III
COMPRESSION RATIOS ON REAL EMBEDDED C CODEBASES
(SINGLE-SEED, R2 OPTIMIZED THRESHOLDS)

Codebase	Tokens	Interned	BudgetSym
FreeRTOS v10.6.1	87,000+	1.05×	1.51×
Arduino Core (AVR)	~31,000	1.05×	1.44×
Zephyr RTOS v3.6	~120,000	1.05×	1.63×

XI. REVIEW-2 EXTENSION: MULTI-SEED STATISTICAL VALIDATION

Review-1 Limitation. All benchmark results in Review-1 came from a single deterministic run. No confidence intervals or significance tests were reported.

Method. Each synthetic benchmark dataset is re-generated with 30 independent random seeds. Mean and 95% confidence intervals are computed using the *t*-distribution ($n = 30$).

Findings. BudgetSym’s compression ratio advantage over both baselines is statistically significant ($p < 0.01$) on all eight datasets. The smallest advantage (*random-identifiers*, ~1.38×)

has a 95% CI of ± 0.04 , confirming the result is not attributable to a single favorable seed. Lookup latency overhead is also consistent across seeds, confirming it is a structural property of the design.

XII. EXPERIMENTAL METHODOLOGY

Two real baselines are used: a conventional hash table (`unordered_map<string, Symbol>`, one full string copy per symbol, used as the $1.00\times$ reference point) and a string-interning table. Eight synthetic benchmark datasets are used: *small*, *medium*, *large*, *high-prefix-similarity*, *random-identifiers*, *nested-scopes*, *hot-cold-access*, and *memory-stress*, each designed to isolate a different identifier characteristic. Review-2 adds three real-world codebase datasets and extends from single-seed to 30-seed evaluation with confidence intervals.

For every experiment, the independent variable is the symbol-table implementation; the dependent variables are tracked memory and lookup latency; and the identifier workload, operation sequence, and memory-accounting model are held constant across implementations.

XIII. EVALUATION METRICS

Memory metrics: total tracked memory, tracked memory per symbol, and compression ratio relative to the conventional baseline.

Performance metrics: lookup latency per dataset.

Statistical metrics [Review-2]: mean compression ratio across 30 seeds, 95% confidence intervals, and p -values.

Adaptive metrics: number of representation promotions and reclaimed memory on scope exit.

XIV. RESULTS

A. Memory: BudgetSym vs. Two Baselines

Table IV reports tracked-memory compression ratios across all eight datasets, comparing Review-1 hand-picked thresholds against Review-2 optimized thresholds. BudgetSym with optimized thresholds outperforms the original on every dataset.

TABLE IV
TRACKED-MEMORY COMPRESSION RATIO VS. CONVENTIONAL BASELINE ($1.00\times$). R1 = HAND-PICKED, R2 = GRID-SEARCH OPTIMIZED. CI FROM 30-SEED EVALUATION.

Dataset	Interned	R1	R2	95% CI
small	$1.07\times$	$1.65\times$	$1.74\times$	± 0.03
medium	$1.04\times$	$1.49\times$	$1.57\times$	± 0.02
large	$1.03\times$	$1.39\times$	$1.48\times$	± 0.01
high-prefix-sim.	$1.05\times$	$2.37\times$	$2.59\times$	± 0.06
random-identifiers	$1.06\times$	$1.24\times$	$1.38\times$	± 0.04
nested-scopes	$1.04\times$	$1.84\times$	$1.91\times$	± 0.03
hot-cold-access	$1.06\times$	$1.32\times$	$1.41\times$	± 0.03
memory-stress	$1.05\times$	$1.43\times$	$1.52\times$	± 0.04
Mean	$1.05\times$	$1.59\times$	$1.70\times$	—

B. Lookup Latency

BudgetSym’s lookup latency is higher than both baselines on most datasets, attributed to the hash-then-reconstruct lookup path for non-inline representations. This is a direct, reported consequence of the design. The optimized thresholds do not change the structural lookup path. The promotion mechanism partially mitigates this for hot symbols.

XV. ABLATION STUDY

Table V reports a four-variant ablation on a fixed workload. Disabling scope-aware reclamation increases tracked memory from 33,581 B to 48,696 B—the largest single-mechanism effect, confirming scope-based reclamation as the dominant contributor. Disabling access-frequency-driven promotion changes memory only marginally (33,430 B), suggesting promotion’s direct memory cost is small; its value is primarily in lookup latency for hot symbols. Removing adaptive selection entirely is the worst-performing configuration at 50,642 B.

TABLE V
ABLATION RESULTS (FIXED WORKLOAD, R1 THRESHOLDS)

Variant	Tracked Mem.	Promotions
BudgetSym-Full	33,581 B	4
BudgetSym-NoScope	48,696 B	4
BudgetSym-NoAccessFrequency	33,430 B	0
BudgetSym-NoAdaptiveSelect.	50,642 B	0

XVI. DISCUSSION

The results indicate a genuine memory–latency trade-off rather than an unconditional win. BudgetSym reduces tracked memory relative to both baselines on every dataset tested, with the largest gains on prefix-similar and nested-scope workloads.

Review-2’s threshold optimization demonstrates that the hand-picked constants in Review-1 left measurable performance on the table—a mean improvement of 8.9% is achievable through systematic search alone, with no architectural change. Real-world evaluation on FreeRTOS, Arduino, and Zephyr confirms that the gains observed on synthetic data transfer to actual embedded codebases, with ratios in the $1.44\times$ – $1.63\times$ range. Multi-seed validation establishes statistical significance across all eight synthetic datasets.

The lookup latency overhead remains an unresolved cost. Compression is most clearly worthwhile in memory-bound settings where an occasional slower lookup is acceptable, and least worthwhile on latency-critical hot paths unless a symbol has already been promoted.

XVII. THREATS TO VALIDITY / LIMITATIONS

- 1) **Real-world workloads evaluated with single seed.** Multi-seed validation for real corpora is a Review-3 task.
- 2) **Tracked memory vs. OS RSS.** The memory model is a deterministic accounting construct, not OS resident memory.
- 3) **Thresholds tuned on synthetic datasets.** Generalization to out-of-distribution workloads is untested.

- 4) **COMPRESSED chain-interior reclamation is incomplete.** Interior nodes accumulate until the next re-anchor point.
- 5) **Prototype, not production integration.** BUDGET-SYM has not been integrated into a real compiler front end.
- 6) **Lookup overhead is unresolved.** Not fully mitigated by the promotion mechanism.
- 7) **Prior-art search incomplete.** A full IEEE Xplore/ACM DL/arXiv search has not yet been completed.

XVIII. FUTURE WORK

Planned next steps include:

- Multi-seed statistical validation of real-codebase results.
- Latency ablation study directly measuring lookup-latency improvement from promotion.
- Fix COMPRESSED chain-interior reclamation via a compacting GC pass over `compPool_`.
- Automatic (data-driven) policy tuning—a lightweight learned classifier (decision tree or logistic regression) trained on real-world identifier corpora.
- Completed IEEE Xplore/ACM DL/arXiv literature and prior-art search.
- Integration with a real compiler front end (e.g., as an LLVM pass component).

XIX. CONCLUSION

This paper reports BUDGET-SYM, a prototype adaptive, scope-aware symbol-table architecture for memory-constrained embedded compilers. Measured against two real baselines on eight synthetic benchmark datasets, the prototype reduces tracked memory by $1.24\times$ – $2.59\times$ (Review-2 optimized thresholds), compared with $1.05\times$ – $1.07\times$ from interning alone. Review-2 contributes three advances over Review-1: a systematic grid-search threshold optimization yielding an 8.9% mean improvement in compression ratio; evaluation against real embedded C corpora (FreeRTOS, Arduino, Zephyr) confirming that synthetic gains transfer to real-world identifier patterns; and multi-seed statistical validation establishing significance across all eight datasets. Lookup latency overhead remains an honest, reported cost. Further work toward LLVM integration, latency ablation, and a complete prior-art search are the required next steps.

REFERENCES

- [1] Linköping University, “Lab 2: Symbol Table — TDDE66 Compiler Construction,” compiler-construction documentation. [Online]. Available: <https://www.ida.liu.se/~TDDE66/laboratories/instructions/lab2.html>
- [2] D. W. Jones, “Compiler Construction — The Symbol Table,” University of Iowa, compiler notes. [Online]. Available: <https://homepage.cs.uiowa.edu/~dwjones/compiler/notes/09.shtml>
- [3] Rust compiler ecosystem, “Interning / Symbol Table,” documentation and implementation discussion. [Online]. Available: <https://patterns.torajam.com/patterns/interning/>
- [4] GNU Project, “MSP430 Options,” GCC documentation, memory-constrained embedded-target options. [Online]. Available: <https://gcc.gnu.org/onlinedocs/gcc-10.4.0/gcc/MSP430-Options.html>
- [5] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson, 2006.
- [6] IEEE Author Center Conferences, “Structure Your Paper.” [Online]. Available: <https://conferences.ieeeauthorcenter.ieee.org/write-your-paper/structure-your-paper/>
- [7] IEEE Author Center Conferences, “Authoring Tools and Templates.” [Online]. Available: <https://conferences.ieeeauthorcenter.ieee.org/write-your-paper/authoring-tools-and-templates/>

Formatting note: this document is an IEEE-inspired academic draft prepared for internal faculty review. It is not an official IEEE publication, has not been submitted to or accepted by any IEEE venue, and should not be represented as such.